System S: A Simple Sized Type System

CS 6117 (Fall 2025)

October 23, 2025

Abstract

These notes describe a simple indexed type system called System S. We first present the syntax and equational theory, before presenting the semantics in two styles: in terms of indexed categories, and in terms of fibered categories. Absolutely nothing in these notes have been reviewed: use at your own risk.

1 Syntax, Type System, and Equational Theory

We will follow the presentation of System F in Chapter 5 of Categories for Types. The main modification is that instead of quantifier over types, as in System F, our types will quantify over *indices* (or if you like, *sizes*). The grammar of types and terms is generated by a pair of signatures $Sg \triangleq (Sg^{ty}, Sg^{tm})$. The signature Sg^{ty} of types and indices is specified by the following data:

- A collection of *index kinds* κ.
- A collection of *index symbols* F, each with an *arity* $F : \kappa_1, \ldots, \kappa_n \to \kappa$ where $n \geq 0$. An index symbol K with arity $n = 0$ is called an *index constant*.
- A collection of *type symbols* α, each with an arity $\alpha : \kappa_1, \ldots, \kappa_n \to Type$

Roughly, we think of the signature Sg^{ty} as the signature for an algebraic theory (as in Chapter 3), except "one level up" since we are now describing things that can appear in types, rather than in terms. Given a countably infinite set of index variables $Var^{in} = \{X, Y, Z, \ldots\}$, the raw indices ($I$) and types ($\Phi$) are given by the grammars:

$$I ::= X \mid K \mid F(I, \ldots, I)$$
$$\Phi ::= \alpha(I, \ldots, I) \mid unit \mid \Phi \times \Phi \mid \Phi \Rightarrow \Phi \mid \forall X : \kappa.\, \Phi.$$

Example 1. Suppose we want indices to represent natural numbers. Then, we could consider a signature Sg^{ty} with one kind Sz representing a size, along with one constant index symbol $Z : Sz$ and one unary index symbol $S : Sz \to Sz$ representing the zero size and the successor of a size, respectively. If we wanted

to, we could add additional index symbols, e.g., $A : Sz, Sz \to Sz$ could be a binary index symbol representing the addition of two sizes. For the types, for every closed type we can have a single type symbol $List_\Phi : Sz \to Type$ for size-indexed lists of elements of type Φ.

Indices and types are well-formed in some index context:

$$\Delta = X_1 \text{ is } \kappa_1, \ldots, X_n \text{ is } \kappa_n.$$

We use the notation X is κ to hopefully distinguish the kind of indices from the type of terms. Then, we can define judgments for well-formed indices and well-formed types as $Sg^{ty} \triangleright \Delta \vdash I$ is κ and $Sg^{ty} \triangleright \Delta \vdash \Phi$ is $Type$, respectively.

The rules for the well-formed essentially follows the rules for well-typed programs from Chapter 3 (only "one level up"):

$$\frac{}{Sg^{ty} \triangleright \Delta, X \text{ is } \kappa, \Delta' \vdash X \text{ is } \kappa} \qquad \frac{K : \kappa \in Sg^{ty}}{Sg^{ty} \triangleright \Delta \vdash K \text{ is } \kappa}$$

$$\frac{F : \kappa_1, \ldots, \kappa_n \to \kappa \in Sg^{ty} \qquad Sg^{ty} \triangleright \Delta \vdash I_1 \text{ is } \kappa_1 \qquad \cdots \qquad Sg^{ty} \triangleright \Delta \vdash I_n \text{ is } \kappa_n}{Sg^{ty} \triangleright \Delta \vdash F(I_1, \ldots, I_n) \text{ is } \kappa}$$

The rules for well-formed types resemble the rules for well-formed types from Figure 5.2 of Chapter 5, except with index variables instead of type variables:

$$\frac{\alpha : \kappa_1, \ldots, \kappa_n \to \kappa \in Sg^{ty} \qquad Sg^{ty} \triangleright \Delta \vdash I_1 \text{ is } \kappa_1 \qquad \cdots \qquad Sg^{ty} \triangleright \Delta \vdash I_n \text{ is } \kappa_n}{Sg^{ty} \triangleright \Delta \vdash \alpha(I_1, \ldots, I_n) \text{ is } Type}$$

$$\frac{}{Sg^{ty} \triangleright \Delta \vdash \mathit{unit} \text{ is } Type} \qquad \frac{Sg^{ty} \triangleright \Delta \vdash \Phi_1 \text{ is } Type \qquad Sg^{ty} \triangleright \Delta \vdash \Phi_2 \text{ is } Type}{Sg^{ty} \triangleright \Delta \vdash \Phi_1 \times \Phi_2 \text{ is } Type}$$

$$\frac{Sg^{ty} \triangleright \Delta \vdash \Phi_1 \text{ is } Type \qquad Sg^{ty} \triangleright \Delta \vdash \Phi_2 \text{ is } Type}{Sg^{ty} \triangleright \Delta \vdash \Phi_1 \Rightarrow \Phi_2 \text{ is } Type}$$

$$\frac{Sg^{ty} \triangleright \Delta, X \text{ is } \kappa \vdash \Phi \text{ is } Type}{Sg^{ty} \triangleright \Delta \vdash \forall X : \kappa. \Phi \text{ is } Type}$$

For terms, the term signature Sg^{tm} is specified by the following data:

- A typing signature Sg^{ty}.
- A collection of function symbols f, each with an arity of the form

  $$f : \Delta; \Phi_1, \ldots, \Phi_n \to \Phi$$

  where $Sg^{ty} \triangleright \Delta \vdash \Phi_i$ is $Type$ for $i = 1, \ldots, n$ and also for Φ. A function symbol k with arity $n = 0$ is called a *constant symbol*; note that such symbols are still allowed to have a non-empty index context context Δ.

The grammar of terms is almost the same as in Chapter 5:

$$\begin{aligned} M ::= {} & x \mid k_{I_1,\ldots,I_n} \mid f_{I_1,\ldots,I_n}(M,\ldots,M) && \text{(variables and symbols)} \\ & \mid () \mid (M,M) \mid fst(M) \mid snd(M) && \text{(unit and pairs)} \\ & \mid \lambda x : \Phi.\, M \mid M\ M && \text{(term abstractions)} \\ & \mid \Lambda X : \kappa.\, M \mid M\ I && \text{(size abstraction)} \end{aligned}$$

We can define a typing judgment $Sg^{tm} \triangleright \Delta \mid \Gamma \vdash M : \Phi$, where $\Gamma = x_1 : \Phi_1, \ldots, x_n : \Phi_n$ is a typing context where the types Φ_i and Φ are well-formed in the index context Δ. The typing rules are essentially the same as Figure 5.4 in Chapter 5, except with index variables instead of type variables. We show the modifications for the size abstraction terms, and omit the rest:

$$\frac{Sg^{tm} \triangleright \Delta, X \text{ is } \kappa \mid \Gamma \vdash M : \Phi}{Sg^{tm} \triangleright \Delta \mid \Gamma \vdash \Lambda X : \kappa.\, M : \forall X : \kappa.\, \Phi}$$

$$\frac{Sg^{tm} \triangleright \Delta \mid \Gamma \vdash M : \forall X : \kappa.\, \Phi \qquad Sg^{ty} \triangleright \Delta \vdash I \text{ is } \kappa}{Sg^{tm} \triangleright \Delta \mid \Gamma \vdash M\ I : \Phi[I/X]}$$

The equational theory is essentially the same as in Figure 5.5-7 in Chapter 5. Briefly, we write $Th \triangleright \Delta \vdash I_1 = I_2$ is κ when two index expressions of kind κ are equal, and $Th \triangleright \Delta \mid \Gamma \vdash M_1 = M_2 : \Phi$ when two well-typed terms of type Φ are equal. The theory Th is parametrized by axioms $Ax = (Ax^{ty}, Ax^{tm})$ for equalities between indices and terms, respectively.

Example 2. Continuing from the previous example, for every closed type Φ, we can add constant and function symbols:

- $nil_\Phi : \cdot\ ;\ \cdot \to List_\Phi(Z)$
- $cons_\Phi : X \text{ is } Sz\ ;\ \Phi, List_\Phi(X) \to List_\Phi(S(X))$

for empty and cons constructors for lists; note that the cons constructor takes a list of size X to a list of size $S(X)$, i.e., one larger.

We might like to define a function symbol for case expressions, but it is not so easy—roughly, the problem is that we would like to track size information (e.g., equalities between size expressions) so that when we are casing on a non-empty list, we can connect the size of the original list to the size of sublist.

2 System S: Indexed Semantics

We first consider a semantics for System S based on indexed categories. We will continue to use notation that is similar to the presentation of System F in Chapter 5 of CfT, but there some key differences. Most importantly, the index layer describes index expressions and kinds, while the type layer describes terms and types. In System F, the index layer describes type expressions and types, so there is a bit more coupling between the two layers.

Our indexed semantics will interpret System S in a functor $\mathbb{C} : \mathbf{Set}^{op} \to \mathbf{CCCat}$, an example of an *indexed category*. Here, **Set** is the ordinary category of sets with some specified choice of finite products, and **CCCat** is the (large) category where objects are Cartesian closed categories and morphisms are functors preserving the ccc structure (products and exponentials). On objects, $\mathbb{C}$ maps each set B to a Cartesian closed category $\mathbb{C}(B)$, which we call the *fiber* over B. On morphisms, the functor $\mathbb{C}$ maps each morphism $f : A \to B$ to a *reindexing* functor in the opposite direction

$$f^* \triangleq \mathbb{C}(f) : \mathbb{C}(B) \to \mathbb{C}(A)$$

This functor is required to preserve the Cartesian closed structure exactly (not just up to isomorphism).

2.1 Modeling System S: Overview

System S combines elements of the languages we've seen in Chapter 3 and Chapter 4 of CfT. Given a type and index signature Sg^{ty}, we can interpret:

- Every index kind κ as a set $[\![\kappa]\!] \in Ob(\mathbf{Set})$.
- Every index symbol $F : \kappa_1, \ldots, \kappa_n \to \kappa$ as a morphism in **Set**:

$$[\![f]\!] : [\![\kappa_1]\!] \times \cdots \times [\![\kappa_n]\!] \to [\![\kappa]\!]$$

- Every type symbol $\alpha : \kappa_1, \ldots, \kappa_n \to Type$ as an object in the fiber:

$$[\![\alpha]\!] \in Ob(\mathbb{C}([\![\kappa_1]\!] \times \cdots \times [\![\kappa_n]\!]))$$

By extending this interpretation, we can interpret:

- Every well-formed index expression $Sg^{ty} \triangleright \Delta \vdash I$ is κ is interpreted as a morphism
$$[\![\Delta \vdash I \text{ is } \kappa]\!] : [\![\Delta]\!] \to [\![\kappa]\!]$$
in **Set**.
- Every well-formed type expression $Sg^{ty} \triangleright \Delta \vdash \Phi$ is $Type$ is interpreted as an object
$$[\![\Delta \vdash \Phi]\!]$$
in the fiber category $\mathbb{C}([\![\Delta]\!])$.

Given an interpretation for the type and index signature Sg^{ty} and given a term signature Sg^{tm}, we interpret every function symbol

$$f : \Delta; \Phi_1, \ldots, \Phi_n \to \Phi$$

as a morphism in the fiber $\mathbb{C}([\![\Delta]\!])$:

$$[\![f]\!] : [\![\Phi_1]\!] \times \cdots \times [\![\Phi_n]\!] \to [\![\Phi]\!].$$

By extending this interpretation, every well-typed term $Sg^{tm} \triangleright \Delta \mid \Gamma \vdash M : \Phi$ is interpreted as a morphism

$$[\![\Delta \mid \Gamma \vdash M : \Phi]\!] : [\![\Delta \vdash \Gamma]\!] \to [\![\Delta \vdash \Phi]\!]$$

in the fiber $\mathbb{C}([\![\Delta]\!])$, where if $\Gamma = x_1 : \Phi_1, \ldots x_n : \Phi_n$ is a typing context, then

$$[\![\Delta \vdash \Gamma]\!] \triangleq [\![\Delta \vdash \Phi_1]\!] \times \cdots \times [\![\Delta \vdash \Phi_n]\!]$$

is an object in $\mathbb{C}([\![\Delta]\!])$, a Cartesian closed category.

If this interpretation validates all axioms in Ax^{tm}, then this interpretation should automatically validate the equational theory of System S. Below, we look more closely at two important aspects of this semantics: (1) how to model substitution of index terms into types and terms, and (2) how to model types and terms for index polymorphism.

2.2 Modeling Index Substitution

We think of each object (set) in the base category **Set** as representing a kind (or a kind context), and we think of morphisms in the base category **Set** as representing an index expression. Objects and morphisms in $\mathbb{C}(B)$ are well-formed types and well-typed terms in the index context B. *Reindexing* along a morphism f models how the types and terms in a context B transform when we substitute an index expression—represented by f—in index context B'.

For example, suppose we have two index contexts $\Delta = X_1$ is $\kappa_1, \ldots, X_m$ is κ_m, and $\Delta' = Y_1$ is $\kappa'_1, \ldots, Y_n$ is κ'_n. If we have m index terms $\Delta' \vdash I_a$ is κ_a for $a = 1, \ldots, m$, then we can pair them to build a morphism

$$i \triangleq \langle [\![\Delta' \vdash I_a \text{ is } \kappa_a]\!] \mid a = 1, \ldots, m \rangle : [\![\Delta']\!] \to [\![\Delta]\!]$$

The reindexing functor $i^* : \mathbb{C}([\![\Delta]\!]) \to \mathbb{C}([\![\Delta']\!])$ transforms both objects and morphisms. Recalling that types are interpreted as objects and terms are interpreted as morphisms, this reindexing operation describes what happens to types and terms when we substitute in index expressions, changing the index context from Δ to Δ':

- On objects, i^* takes (the semantics of) the type $\Delta \vdash \Phi$ to (the semantics of) the type $\Delta' \vdash \Phi[I/\Delta]$.
- On morphisms, i^* takes (the semantics of) the term $\Delta \mid \Gamma \vdash M : \Phi$ to (the semantics of) the type $\Delta' \mid \Gamma[I/\Delta] \vdash M[I/\Delta] : \Phi[I/\Delta]$.

Here, we write $[I/\Delta]$ for the substitution of index expressions I_a for index variables X_a, and for typing context we write:

$$\Gamma[I/\Delta] \triangleq x_1 : \Phi_1[I/\Delta], \ldots, x_n : \Phi_n[I/\Delta].$$

The properties above are not assumed as part of the semantics, but rather they can be proved by induction on the derivation of types and terms—they

follow from two new substitution lemmas for System S, in addition to the usual substitution lemma we've seen before stating that term substitution is modeled by composition; note that term substitution can only happen in a single index context Δ, so it makes sense to compose morphisms since they belong to the same category $\mathbb{C}(\llbracket C \rrbracket)$. We will not spell these lemmas out, but they are similar to Lemmas 5.4.2, 5.4.7, and 5.4.9 in CfT.

2.3 Modeling Index Polymorphism

To model index polymorphism, we need to define the semantics of:

1. A new type for universal quantification: $\forall X : \kappa.\, \Phi$.
2. A new term for index abstraction: $\Lambda X : \kappa.M$.
3. A new term for index application: $M\ [I]$.

Our semantics must also validate the equational theory for index abstraction (β, η, and ξ equalities, just like we saw for STLC), but these are a bit technical to show so we will not detail soundness (Theorem 5.4.12 in CfT spells out the details).

Let's think through how we might model these three components. For types, the type formation rule looks like:

$$\frac{\Delta, X \text{ is } \kappa \vdash \Phi \text{ is } Type}{\Delta \vdash \forall X : \kappa.\, \Phi \text{ is } Type}$$

Based on what we have said before, the premise is modeled as an object in $\mathbb{C}(\llbracket \Delta, X \text{ is } \kappa \rrbracket) = \mathbb{C}(\llbracket \Delta \rrbracket \times \llbracket \kappa \rrbracket)$, and the conclusion is modeled as an object in $\mathbb{C}(\llbracket \Delta \rrbracket)$. Thus, the first thing we need is a map:

$$\forall : Ob(\mathbb{C}(\llbracket \Delta \rrbracket \times \llbracket \kappa \rrbracket)) \to Ob(\mathbb{C}(\llbracket \Delta \rrbracket))$$

It is natural to wonder whether this map should be extended to a functor: do we want to map morphisms to morphisms too? We can take a look at the typing rule for introducing index abstractions:

$$\frac{\Delta, X \text{ is } \kappa \mid \Gamma \vdash M : \Phi \qquad X \notin fiv(\Gamma)}{\Delta \mid \Gamma \vdash \Lambda X : \kappa.\, M : \forall X : \kappa.\, \Phi}$$

The side condition is very important: it states that the index variable X *must not appear free* in the types of Γ. Syntactically, this condition is needed to ensure that the conclusion is a well-formed typing judgment: types in Γ are only allowed to mention index variables in Δ, not X. This side condition also turns out to be crucial semantically, as we will see next.

But for now, we note that the premise is interpreted as a morphism in $\mathbb{C}(\llbracket \Delta \rrbracket \times \llbracket \kappa \rrbracket)$, while the conclusion is interpreted as a morphism in $\mathbb{C}(\llbracket \Delta \rrbracket)$, so it looks like we do want a functor

$$\forall : \mathbb{C}(\llbracket \Delta \rrbracket \times \llbracket \kappa \rrbracket) \to \mathbb{C}(\llbracket \Delta \rrbracket)$$

But we will also want a few more properties of this functor in order to capture universal quantification.

To motivate why, we first consider how index variables can be added or removed from the context. There is a projection morphism $\pi : [\![\Delta]\!] \times [\![\kappa]\!] \to [\![\Delta]\!]$ and a reindexing functor $\pi^* : \mathbb{C}([\![\Delta]\!]) \to \mathbb{C}([\![\Delta]\!] \times [\![\kappa]\!])$. Intuitively, this reindexing functor (also known as *weakening*) takes the object interpreting the type $\Delta \vdash \Phi$ and maps it to the object interpreting the type $\Delta, X \text{ is } \kappa \vdash \Phi$, that is, the same syntactic type but in a context extended with more index variables (that don't appear in the type). Likewise, this reindexing functor takes the morphism interpreting the term $\Delta \mid \Gamma \vdash M : \Phi$ to the morphism interpreting the term $\Delta, X \text{ is } \kappa \mid \Gamma \vdash M : \Phi$, again extending the index context with an additional (unused) index variable.

We can now try to interpret the index abstraction typing rule again, using these functors:

$$\frac{[\![\Delta, X \text{ is } \kappa \mid \Gamma \vdash M : \Phi]\!] : \pi^*([\![\Delta \vdash \Gamma]\!]) = [\![\Delta, X \text{ is } \kappa \vdash \Gamma]\!] \to [\![\Delta, X \text{ is } \kappa \vdash \Phi]\!]}{[\![\Delta \mid \Gamma \vdash \Lambda X : \kappa.\, M : \forall X : \kappa.\, \Phi]\!] : [\![\Delta \vdash \Gamma]\!] \to [\![\Delta \vdash \forall X : \kappa.\, \Phi]\!] = \forall([\![\Delta, X \text{ is } \kappa \vdash \Phi]\!])}$$

The equality $\pi^*([\![\Delta \vdash \Gamma]\!]) = [\![\Delta, X \text{ is } \kappa \vdash \Gamma]\!]$ precisely captures the side condition we mentioned earlier: since X does not appear in the types of Γ, the types in Γ are must be well-formed in the smaller context Δ.

Now, we might recognize that the above inference rule looks a bit like an adjunction $\pi^* \dashv \forall$, that is, universal quantification should be *right* adjoint to weakening:

$$\overline{(-)} : \mathbb{C}([\![\Delta]\!] \times [\![\kappa]\!])(\pi^*(\psi), \phi) \rightleftarrows \mathbb{C}([\![\Delta]\!])(\psi, \forall(\phi)) : \widehat{(-)}$$

Finally, we will require that the *Beck Chevalley* condition holds: for every morphism $f : A' \to A$ in **Set** the following diagram commutes

$$\begin{array}{ccc} \mathbb{C}(A \times B) & \xrightarrow{\forall_{A,B}} & \mathbb{C}(A) \\ \Big\downarrow{\scriptstyle (f \times id_B)^*} & & \Big\downarrow{\scriptstyle f^*} \\ \mathbb{C}(A' \times B) & \xrightarrow{\forall_{A',B}} & \mathbb{C}(A') \end{array}$$

and the "canonical natural transformation" described at the end of Chapter 5.3 in CfT is the identity. Roughly, the vertical morphisms encode substitution for index expressions, while the horizontal morphisms encode index abstraction; thus, index abstraction commutes with index substitution.

Thus, we have enough to interpret the index abstraction rule:

$$\frac{[\![\Delta, X \text{ is } \kappa \mid \Gamma \vdash M : \Phi]\!] \triangleq m : \pi^*([\![\Delta \vdash \Gamma]\!]) \to [\![\Delta, X \text{ is } \kappa \vdash \Phi]\!]}{[\![\Delta \mid \Gamma \vdash \Lambda X : \kappa.\, M : \forall X : \kappa.\, \Phi]\!] \triangleq \overline{m} : [\![\Delta \vdash \Gamma]\!] \to \forall([\![\Delta, X \text{ is } \kappa \vdash \Phi]\!])}$$

where we transpose the morphism m interpreting the premise across the adjunction to get a morphism $\overline{m}$ interpreting the conclusion.

Requiring an adjunction actually gives more: we can also interpret the rule in the reverse direction since there is a bijective correspondence between morphisms on the top and on the bottom; we won't use this bijection here but it is important for showing that the equations hold, i.e., that we can convert a morphism interpreting a program of forall type into a morphism interpreting a program of non-forall type in a larger index context (β equality) and that all morphisms interpreting a program of forall type arise in this way (η equality).

But for now, we will just consider how to interpret index application. This rule is more complicated to interpret since it involves (1) getting rid of a forall type ("instantiating"), and (2) substituting an index expression. We first describe the unit and counit morphisms (actually, natural transformations) of the adjunction. The unit arises as follows:

$$\overline{(-)} : \mathbb{C}([\![\Delta]\!] \times [\![\kappa]\!])(\pi^*(\psi), \pi^*(\psi)) \to \mathbb{C}([\![\Delta]\!])(\psi, \forall(\pi^*(\psi)))$$

Then the unit η is $\overline{id}$. Intuitively, it is a morphism that allows converting a program of type Ψ into a program of type $\forall X : \kappa.\, \Psi$, where X does not appear in Ψ.

The counit is more useful for our purposes. It arises from the applying the adjunction in the reverse direction:

$$\mathbb{C}([\![\Delta]\!] \times [\![\kappa]\!])(\pi^*(\forall(\phi)), \phi) \leftarrow \mathbb{C}([\![\Delta]\!])(\forall(\phi), \forall(\phi)) : \widehat{(-)}$$

Then the counit ϵ is $\widehat{id}$. Intuitively, it is a morphism that allows converting a program of type $\forall X : \kappa.\, \Phi$ to a program of type $\Phi[Y/X]$, where Y appears in the context Δ. That is, it *instantiates* a forall type with an index variable in the current index context. By combining the counit with reindexing, we will will be able to instantiate a forall type with any (well-formed) index expression, not just an index variable.

Finally, we have everything we need to interpret the index application rule:

$$\frac{[\![\Delta \vdash I \text{ is } \kappa]\!] \triangleq i \qquad [\![\Delta \mid \Gamma \vdash M : \forall X : \kappa.\, \Phi]\!] \triangleq m}{[\![\Delta \mid \Gamma \vdash M\ [I] : \Phi[I/X]]\!] \triangleq \langle id_{[\![\Delta]\!]}, i\rangle^*(\epsilon) \circ m}$$

To understand how the conclusion works, we can spell out the objects. The counit instantiates with a new index variable:

$$[\![\Delta, Y \text{ is } \kappa \vdash \forall X : \kappa.\, \Phi]\!] \xrightarrow{\epsilon} [\![\Delta, Y \text{ is } \kappa \vdash \Phi[Y/X]]\!]$$

Reindexing performs index substitution for the new index variable:

$$[\![\Delta \vdash \forall X : \kappa.\, \Phi]\!] = [\![\Delta \vdash (\forall X : \kappa.\, \Phi)[I/Y]]\!] \xrightarrow{\langle id_{[\![\Delta]\!]}, i\rangle^*(\epsilon)} [\![\Delta \vdash \Phi[Y/X][I/Y]]\!] = [\![\Delta \vdash \Phi[I/X]]\!]$$

And then finally composition gives:

$$[\![\Delta \vdash \Gamma]\!] \xrightarrow{m} [\![\Delta \vdash \forall X : \kappa.\, \Phi]\!] \xrightarrow{\langle id_{[\![\Delta]\!]}, i\rangle^*(\epsilon)} [\![\Delta \vdash \Phi[I/X]]\!]$$

which is exactly the desired type of a morphism to interpret the conclusion of the index application rule.

2.4 Example: A More Concrete Model

We will give a slightly more concrete example of an indexed category. Let $\mathbf{D}$ be any Cartesian closed category, and suppose that $\mathbf{D}$ has *small* (also known as Set-indexed) products—namely, for any set of objects $\{X_a\}_{a \in A}$, there is a product object $\Pi_{a \in A} X_a$.

Then, we can define an indexed category where the functor $\mathbb{C}$ maps each object (set) B in the base to the product category $\mathbf{D}^B$; this category is also Cartesian closed, e.g., it is the functor category $[\mathbf{B}, \mathbf{D}]$ where $\mathbf{B}$ is the set B regarded as a discrete category. The functor $\mathbb{C}$ maps each morphism $f : A \to B$ to a *reindexing* functor in the opposite direction

$$f^* \triangleq \mathbb{C}(f) : \mathbb{C}(B) \to \mathbb{C}(A)$$

where on objects this functor maps

$$f^*((X_b)_{b \in B})_a \triangleq X_{f(a)}$$

and on morphisms this functor maps

$$f^*((\phi_b : X_b \to Y_b)_{b \in B})_a \triangleq \phi_{f(a)} : X_{f(a)} \to Y_{f(a)}.$$

This functor preserves the Cartesian closed structure.

We can also describe the right-adjoint to weakening, used to interpret universal quantification. Explicitly, the functor $\forall_{A,B} : \mathbb{C}(A \times B) \to \mathbb{C}(A)$ is defined on objects as:

$$\forall_{A,B}(X_{(i,j) \in A \times B})_a \triangleq (\Pi_{a \in A} X_{a,b})_{b \in B}$$

and on morphisms as

$$\forall_{A,B}(\phi_{(i,j) \in A \times B} : X_{i,j} \to Y_{i,j})_a \triangleq (\Pi_{a \in A} \phi_{a,b})_{b \in B} : \forall(X)_a \to \forall(Y)_a.$$

Note that this functor uses Set-indexed products in $\mathbf{D}$. By unfolding definitions, it is possible to verify that this functor satisfies the Beck Chevalley conditions.

3 System S: Fibered Semantics

The indexed semantics above interprets the language given an indexed category, i.e., a functor $\mathbb{C} : \mathbf{Set}^{op} \to \mathbf{CCCat}$. There is an alternative way of encoding this indexed category as a *fibered category* (also sometimes called a *fibration*), a functor $p : \mathbf{E} \to \mathbf{Set}$ with special properties. The benefits of this approach are that (1) $\mathbf{E}$ and $\mathbf{C}$ are both standard categories unlike $\mathbf{CCCat}$, which is a category but in some sense one level higher, (2) fibered categories are more general than indexed categories, and there are more natural examples of fibered categories, and (3) there are more ways of combining and building fibrations; for instance fibrations compose via functor composition, so we can modularly define a semantics for languages with more layers than we consider here.

Intuitively, the reason that indexed categories and fibered categories are equivalent (in some sense we will not make precise) is the same reason that $[\mathbf{B}, \mathbf{Set}]$ is equivalent to $\mathbf{Set}/B$, which you have already seen on your homework: we can express the fact that some structure depends on another structure B either by encoding a map (functor) *out of* B, or encoding a map (functor) *into* B. The first kind of encoding is usually called *indexed*, while the second kind of encoding is usually called *fibered* or *displayed*.

Fibered category theory is a rich corner of category theory for talking about a category $\mathbf{E}$ *relative to* some base category $\mathbf{B}$, and there is no way we can cover it here. (Bart Jacobs book, "Categorical Logic and Type Theory," is nearly 800 pages of categorical logic built on fibered categories as the main unifying concept.) We will just work through one concrete example: the family fibration $p : Fam(\mathbf{D}) \to \mathbf{Set}$. Although it may look a bit strange initially, this is one of the main motivating examples of a fibration.

The category $Fam(\mathbf{D})$ is defined as follows:

Objects: Pairs $(\{X_i\}_{i \in I}, I)$, where $X_i \in Ob(\mathbf{D})$ and I is a set. Intuitively: tuples of I-many objects of $\mathbf{D}$ (paired with the indexing set I).

Morphisms: a morphism $(\{X_i\}_{i \in I}, I) \to (\{Y_j\}_{j \in J}, J)$ consists of a (set) function $u : I \to J$, and a tuple of morphisms $\{f_i : X_i \to Y_{u(i)}\}_{i \in I}$ in $\mathbf{D}$.

The functor $p : Fam(\mathbf{D}) \to \mathbf{Set}$ simply extracts the indices:

- The object $(\{X_i\}_{i \in I}, I)$ is mapped to I.
- The morphism $(\{f_i\}_{i \in I}, u)$ is mapped to u.

To show that p is a fibration, we must define how to lift morphisms u from the base category $\mathbf{Set}$ to a morphism in the category $Fam(\mathbf{D})$ above u. Formally, given any morphism $u : I \to J$ and any object $(\{X_j\}_{j \in J}, J)$ sitting above the codomain of u, we can define the object in $Fam(\mathbf{D})$

$$u^*(\{X_j\}_{j \in J}, J) \triangleq (\{X_{u(i)}\}_{i \in I}, I)$$

and the morphism in $Fam(\mathbf{D})$:

$$(\{f_i : X_{u(i)} \to X_{u(i)}\}_{i \in I}, u) : u^*(\{X_j\}_{j \in J}, J) \to (\{X_j\}_{j \in J}, J)$$

where all the f_i are the identity morphisms.

It turns out that this lifting is the "best possible lifting" of u, in the sense that any other lifting of a map factoring through u to a morphism in $Fam(\mathbf{D})$ with the specified codomain $(\{X_j\}_{j \in J}, J)$ will factor through our lifting; in other words, our lifting is a *Cartesian* lifting of u. Thus every morphism $u : I \to J$ in the base category has a Cartesian lift, which is what it means for $p : Fam(\mathbf{D}) \to \mathbf{Set}$ to be a (Grothendieck) fibration.

The fibration $p : Fam(\mathbf{D}) \to \mathbf{Set}$ encodes much of the same data we saw in our indexed category $\mathbb{C} : \mathbf{Set}^{op} \to \mathbf{CCCat}$, just in a different form. For instance, consider any set A in the base, we can define the *fiber* category over A, written $Fam(\mathbf{D})_A$:

Objects: Pairs $(\{X_i\}_{i\in A}, A)$, where $X_i \in Ob(\mathbf{D})$. These are the objects in $Fam(\mathbf{D})$ sitting over A.

Morphisms: A morphism from $(\{X_i\}_{i\in A}, A) \to (\{Y_j\}_{j\in A}, A)$ is of the form $(\{f_i : X_i \to Y_i\}_{i\in A}, id_A)$. These morphisms are also called *vertical* morphisms because they are above the identity morphism in the base.

Fiber categories can be defined for any fibration, not just the family fibration. If $\mathbf{D}$ is Cartesian closed, then each fiber $Fam(\mathbf{D})_A$ is also Cartesian closed; thus, the fibration encodes the object part of the functor $\mathbb{C} : \mathbf{Set}^{op} \to \mathbf{CCCat}$.

Moreover, any set map $u : I \to J$ in the base induces a functor $u^* : Fam(\mathbf{D})_J \to Fam(\mathbf{D})_I$; note that this functor is in the *opposite* direction, and this functor is sometimes called *re-indexing* since it changes the index from J to I. The object part of the functor u^* is easy to describe: we let

$$u^*(\{X_j\}_{j\in J}, J) \triangleq (\{X_{u(i)}\}_{i\in I}, I)$$

just like before. For the morphism part of the functor, let $f : (\{X_i\}_{i\in J}, J) \to (\{Y_j\}_{j\in J}, J)$ be any (vertical) morphism in $Fam(\mathbf{D})_J$. We can lift u to the codomain of f, giving a Cartesian morphism

$$\overline{u}_1 : u^*(\{Y_j\}_{j\in J}, J) \to (\{Y_j\}_{j\in J}, J)$$

sitting over $u : I \to J$. But we can also lift u to the *domain* of f, giving a morphism

$$\overline{u}_2 : u^*(\{X_j\}_{j\in J}, J) \to (\{X_j\}_{j\in J}, J)$$

Now (it might be useful to draw some pictures) the composition $f \circ \overline{u}_2$ is a morphism over u with codomain $(\{Y_j\}_{j\in J}, J)$, so it must factor through $\overline{u}_1$, i.e., there must be a (unique) vertical morphism

$$u^*(f) : u^*(\{X_j\}_{j\in J}, J) \to u^*(\{Y_j\}_{j\in J}, J)$$

such that $\overline{u}_1 \circ u^*(f) = f \circ \overline{u}_2$. This $u^*(f)$ is the action of u^* on morphism f.

But we still need to check that $u^* : Fam(\mathbf{D})_J \to Fam(\mathbf{D})_I$ is actually a functor, namely that it maps identities to identities and satisfies the composition law for morphisms. For the family fibration $p : Fam(\mathbf{D}) \to \mathbf{Set}$, this can be checked directly. But for general fibrations $p : \mathbf{E} \to \mathbf{B}$, the situation is more subtle—in order for the reindexing u^* to satisfy the functor laws, the Cartesian lifts must be chosen so that the lift of every composition $v \circ u$ in the base $\mathbf{B}$ is equal to the composition of the lifts of v and u in the total category $\mathbf{E}$.

This kind of fibration is called a *split* fibration, and it plays an important role throughout semantics because reindexing is often modeling some kind of substitution, and substitution is associative and identity preserving. Since many fibrations occurring naturally are not split, there has been a huge amount of work on how to convert general fibrations into equivalent split fibrations. For instance, it is known that every fibration is equivalent to a split fibration, but here the meaning of "equivalence" also becomes more subtle since it means equivalence of *fibrations*, not equivalence of categories.

Finally, the functor u^* for the family fibration also preserves the Cartesian closed structure on each fiber $Fam(\mathbf{D})_A$. That is, it maps products to products and exponentials to exponentials. While a Cartesian closed functor is only required to preserve products and exponentials up to (a specific) isomorphism, the re-indexing functors u^* for the family fibration preserve the Cartesian closed structure exactly, *on-the-nose*.

This completes our description of how the family fibration $p : Fam(\mathbf{D}) \to \mathbf{Set}$ encodes an indexed category $\mathbb{C} : \mathbf{Set}^{op} \to \mathbf{CCCat}$. It is also possible to go the other way, turning an indexed category into a fibration via the Grothendieck construction, which uses the indexed category to define an appropriate total category $\mathbf{E}$ over $\mathbf{Set}$. This construction is somewhat technical, but in our case of a $\mathbf{Set}$-indexed category, it produces something like the family fibration.